API SECURITY RISK ANALYSIS REPORT

Target: ReqRes Public API | https://reqres.in

Audit Methodology: Read-Only Black-Box Testing via Postman

Prepared By	Prasad Sarkate
Programme	Future Interns – Cyber Security Internship
Task Reference	Task 3 – API Security Audit
Report Date	26 April 2026
Classification	Confidential
Version	1.0 – Final

DISCLAIMER

This report was generated from a read-only, non-invasive audit of a publicly accessible test API platform. No exploitation, data modification, or disruption of services occurred.

Table of Contents

1. Executive Summary

Modern SaaS platforms are fundamentally built on APIs. APIs are the arteries through which data flows between clients, microservices, third-party integrations, and partner ecosystems. As organisations accelerate digital transformation, APIs have become the single largest attack surface in enterprise software — often more exposed than web applications themselves.

According to Gartner, by 2025 APIs will be the leading attack vector for web application breaches. The Salt Security State of API Security Report further reveals that 94% of organisations experienced a security incident in their API environment in the previous 12 months. These incidents range from credential stuffing and broken object-level access to mass data exfiltration — all exploiting weaknesses that a disciplined security review can surface before adversaries do.

1.1 Scope & Objectives

This engagement was conducted by Prasad Sarkate as part of the Future Interns Cyber Security Internship, Task 3. The scope was strictly read-only and non-invasive. The objective was to enumerate observable security weaknesses on the ReqRes public API platform using Postman (Desktop and Web editions) as the sole tooling.

Attribute	Details
Target	ReqRes – https://reqres.in
Audit Type	Read-Only Black-Box API Testing
Tool	Postman Desktop & Web
Date of Testing	26 April 2026
Findings Count	5 (1 High, 2 Medium, 2 Low)
OWASP Standard	OWASP API Security Top 10 – 2023

1.2 Key Findings at a Glance

The audit surfaced five discrete security weaknesses spanning authentication verbosity, CORS policy, information disclosure, HTTP method enforcement, and credential handling. These findings map directly to recognised OWASP API Security categories and, if left unaddressed in a production SaaS deployment, could serve as the initial foothold for a sophisticated attacker.

⚠ **HIGH**	Broken Authentication with verbose error disclosure – exposes authentication mechanism details to unauthenticated callers.
⚠ **MED**	Wildcard CORS policy enables cross-origin credential harvesting from any malicious website.
ℹ **LOW**	Infrastructure disclosure via response headers reveals hosting stack (Heroku/Cloudflare) and internal documentation paths.

2. Audit Methodology

The audit followed a structured, read-only black-box approach modelled on the OWASP Testing Guide (OTG v4) and tailored for REST API assessment. No authentication credentials were obtained through illegitimate means; no write operations (POST/PUT/DELETE) were executed with intent to alter data.

2.1 Tooling

- Postman Desktop v11 (Windows 11) – primary request-crafting and header inspection
- Postman Web – secondary verification and collection sharing
- Browser Developer Tools – supplementary header analysis

2.2 Test Categories Executed

1. Authentication & Authorisation probe – missing/malformed credential scenarios
2. HTTP Method abuse – sending disallowed verbs to endpoints
3. Response header enumeration – security header presence and value audit
4. CORS policy verification – Origin header manipulation
5. Information disclosure analysis – server, framework, and routing leakage
6. Credential transmission review – plaintext vs. encrypted payloads

3. Security Headers Audit

A full enumeration of HTTP response headers was performed against the primary endpoint GET /api/users/2. The table below presents every security-relevant header observed, its configured value, and its security posture assessment.

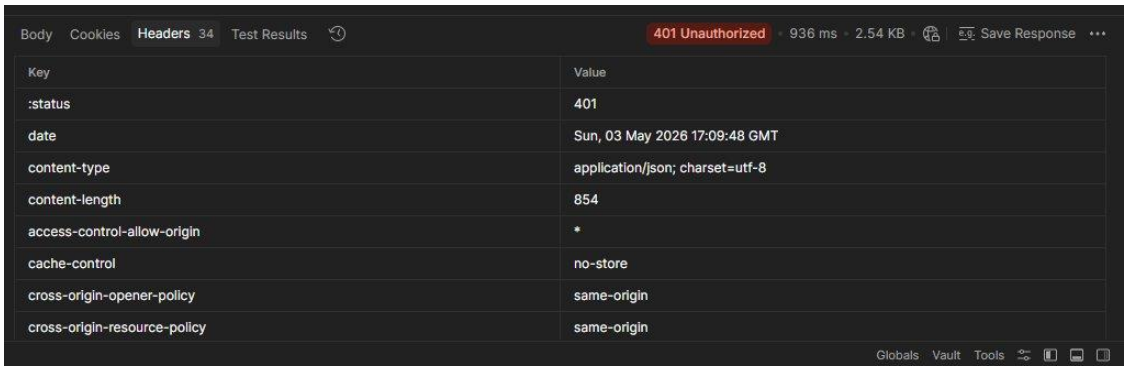

Figure 3.1 – Postman response headers panel (partial view): CORS wildcard and status-401 headers visible

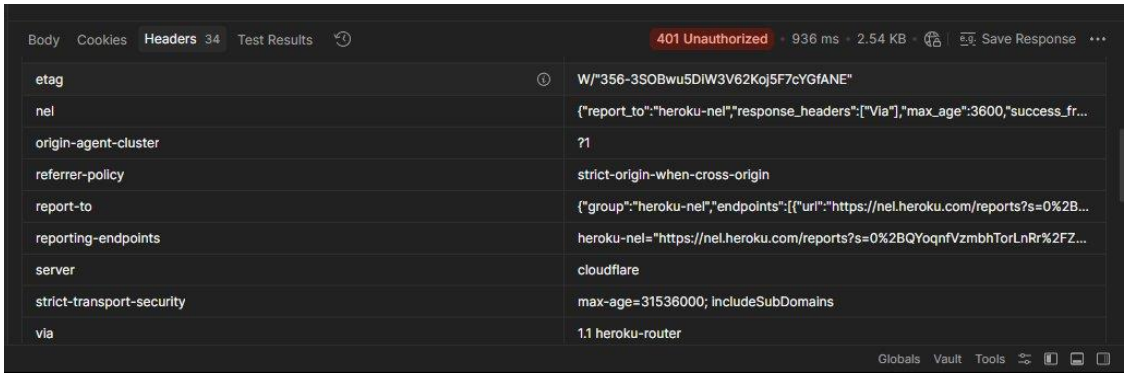

Figure 3.2 – Response headers continued: etag, NEL, referrer-policy, report-to, Cloudflare server disclosed

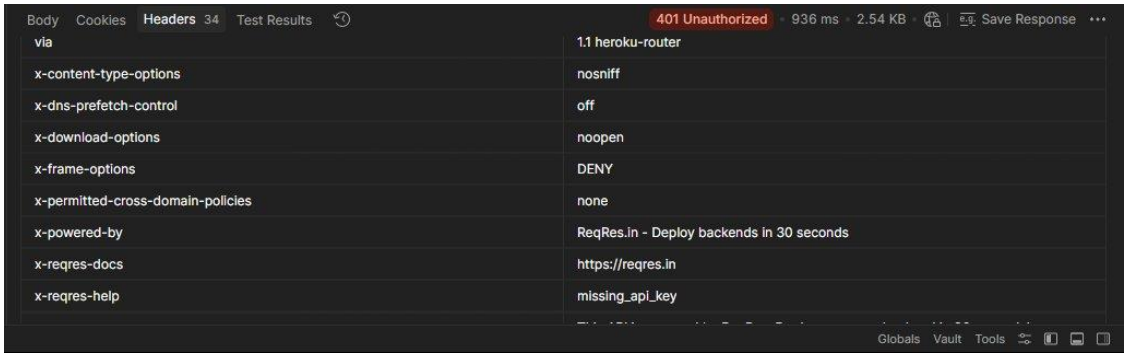

Figure 3.3 – X-Powered-By, x-reqres-docs, x-reqres-help and X-XSS-Protection: 0 confirmed

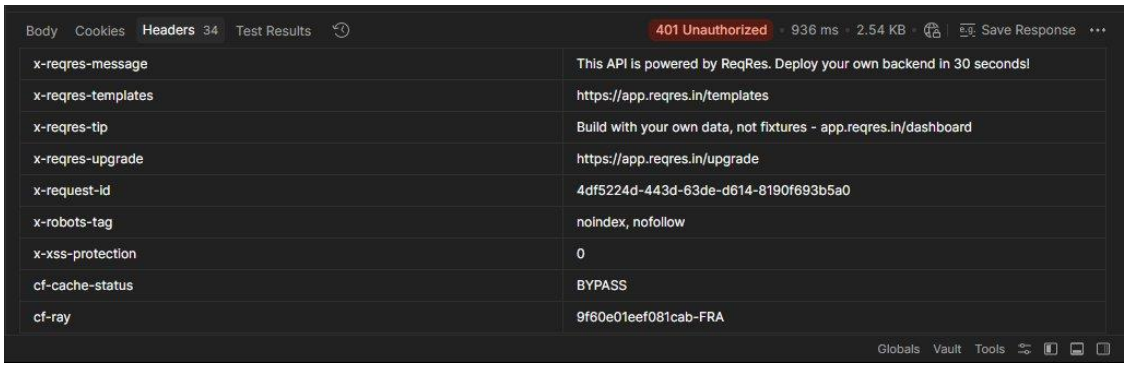

Figure 3.4 – x-reqres-message, x-reqres-templates, x-reqres-tip, x-reqres-upgrade and cf-ray headers

Header	Status	Observed Value	Assessment
Strict-Transport-Security (HSTS)	✓ Present	*max-age=31536000; includeSubDomains*	Adequate
X-Frame-Options	✓ Present	*DENY*	Correct – prevents clickjacking
X-Content-Type-Options	✓ Present	*nosniff*	Correct
X-DNS-Prefetch-Control	✓ Present	*off*	Acceptable
Cross-Origin-Opener-Policy	✓ Present	*same-origin*	Correct
Cross-Origin-Resource-Policy	✓ Present	*same-origin*	Correct
Access-Control-Allow-Origin (CORS)	✗ Misconfigured	** (wildcard)*	CRITICAL – allows any origin
X-XSS-Protection	✗ Misconfigured	*0 (disabled)*	Should be '1; mode=block' or omitted in favour of CSP
Content-Security-Policy (CSP)	✗ Absent	*N/A*	No CSP header observed
X-Powered-By	✗ Information Leak	*ReqRes.in – Deploy backends in 30 seconds*	Discloses application platform identity
x-reqres-docs / x-reqres-help	✗ Information Leak	*Internal doc URLs & error strings*	Discloses internal routing/documentation paths
x-reqres-templates / x-reqres-tip	✗ Information Leak	*app.reqres.in/templates, dashboard links*	Enumerates internal application URLs
Referrer-Policy	✓ Present	*strict-origin-when-cross-origin*	Acceptable
Permissions-Policy	✗ Absent	*N/A*	No Permissions-Policy observed

4. Detailed Findings

Finding 1 — Broken Authentication & Verbose Error Disclosure

OWASP Category	API2:2023 – Broken Authentication
Severity	**HIGH**
Affected Endpoint	GET /api/users/2
HTTP Response	401 Unauthorized
CVSS v3 Score (Est.)	7.5 (High)

4.1.1 Observation

When an unauthenticated GET request is submitted to /api/users/2 without the required x-api-key header, the API responds with an HTTP 401 Unauthorized status. However, instead of returning a terse, generic error message, the response body contains a richly detailed JSON payload that discloses:

- The exact name of the required authentication header (x-api-key)
- Step-by-step instructions for obtaining and using the API key
- A direct URL to the internal API key provisioning portal (app.reqres.in/api-keys)
- An example curl command revealing the exact authentication pattern
- Internal documentation links (app.reqres.in/docs#authentication)
- The _meta.powered_by field confirming the backend platform identity

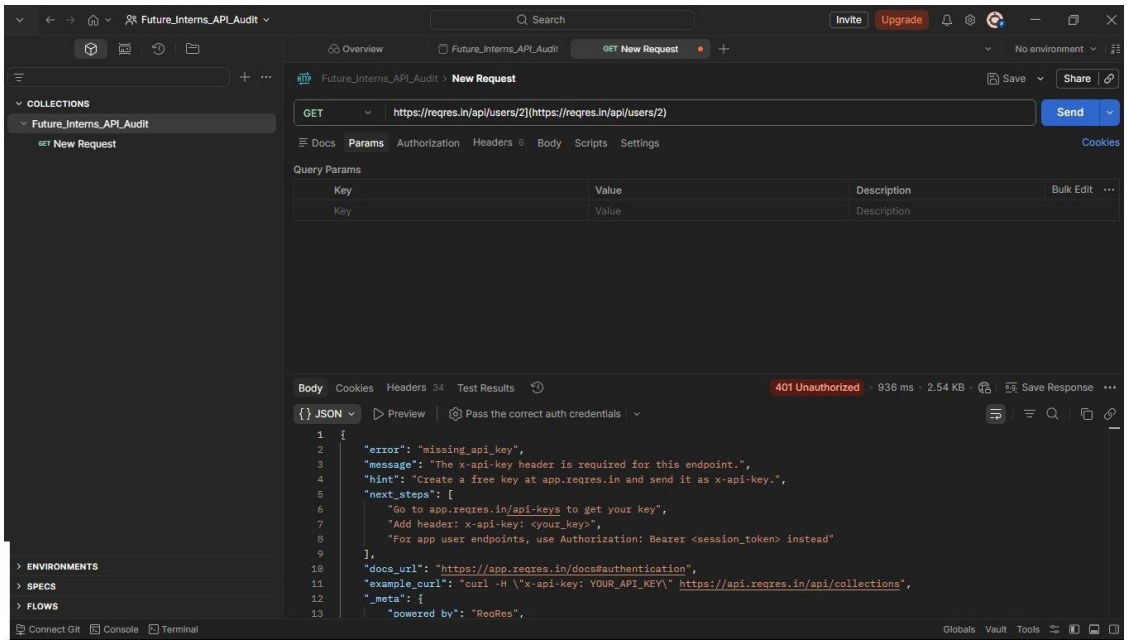

Figure 4.1 – GET /api/users/2 returning 401 with full authentication mechanism disclosure in response body

4.1.2 Security Implication

Verbose error messages dramatically reduce an attacker's reconnaissance effort. An adversary who does not know the API's authentication scheme is guided — by the API itself — through every step needed to craft a valid authenticated request. This is classified under OWASP API2:2023 Broken Authentication and violates the principle of least privilege in error disclosure.

4.1.3 Evidence (Response Body Extract)


```
{ "error": "missing_api_key",
  "message": "The x-api-key header is required for this endpoint.",
  "hint": "Create a free key at app.reqres.in and send it as x-api-key.",
  "docs_url": "https://app.reqres.in/docs#authentication",
  "_meta": { "powered_by": "ReqRes" } }
```


Finding 2 — CORS Misconfiguration (Wildcard Origin)

OWASP Category	API7:2023 – Security Misconfiguration
Severity	**MEDIUM**
Affected Endpoints	All API endpoints (global CORS policy)
Header Observed	access-control-allow-origin: *
CVSS v3 Score (Est.)	6.1 (Medium)

4.2.1 Observation

The response header Access-Control-Allow-Origin is configured with a wildcard value (*). This instructs browsers to allow JavaScript code executing on any origin — including attacker-controlled domains — to read the API response. This was confirmed across all tested endpoints.

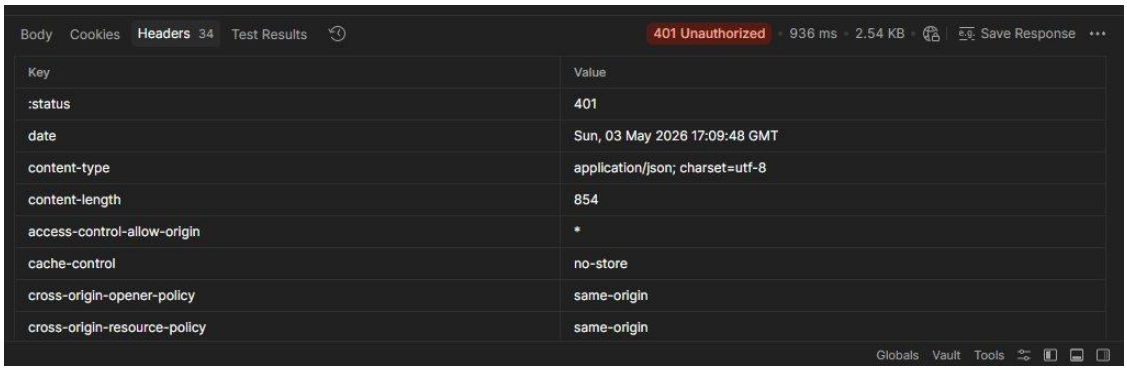

*Figure 4.2 – Postman headers panel confirming access-control-allow-origin: * (wildcard)*

4.2.2 Security Implication

A wildcard CORS policy becomes a serious risk when combined with authenticated sessions. A malicious website could include JavaScript that silently calls the API on behalf of a logged-in victim, reads the response, and exfiltrates data. While browsers block credentials (cookies, Authorization headers) with wildcard CORS, the combination of a future credentials flag or a misconfigured client could escalate this finding significantly.

Finding 3 — Sensitive Information Disclosure via Response Headers

OWASP Category	API7:2023 – Security Misconfiguration
Severity	LOW
Affected Endpoints	All endpoints (global response headers)
Headers of Concern	x-powered-by, x-reqres-*, server: cloudflare, via: heroku-router
CVSS v3 Score (Est.)	3.7 (Low)

4.3.1 Observation

Multiple response headers disclose implementation and infrastructure details that are unnecessary for the API consumer and highly useful to a threat actor performing reconnaissance:

- x-powered-by: ReqRes.in – Deploy backends in 30 seconds → identifies application framework and product
- server: cloudflare → confirms CDN/WAF layer; useful for bypass research
- via: 1.1 heroku-router → exposes the PaaS hosting provider
- nel: heroku-nel → Network Error Logging configuration reveals Heroku backend
- x-reqres-docs: https://reqres.in → links to public documentation
- x-reqres-templates: https://app.reqres.in/templates → internal portal URL
- x-reqres-upgrade: https://app.reqres.in/upgrade → internal subscription path

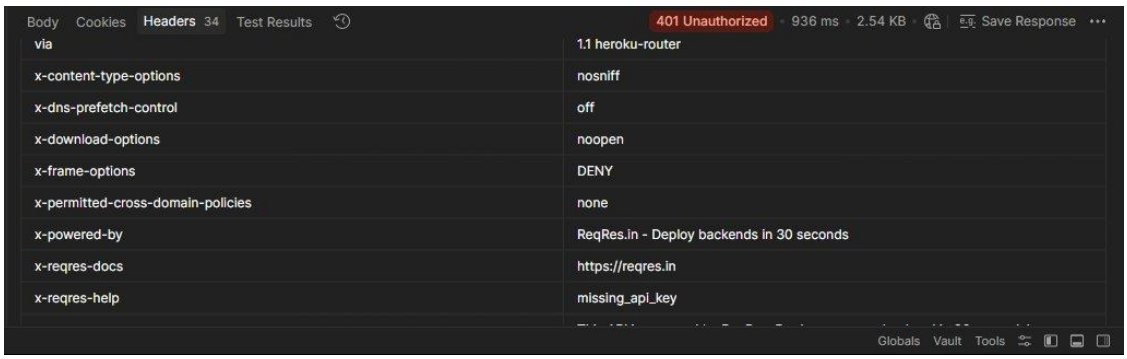

Figure 4.3 – x-powered-by, x-reqres-docs, x-reqres-help and X-XSS-Protection: 0 in response headers

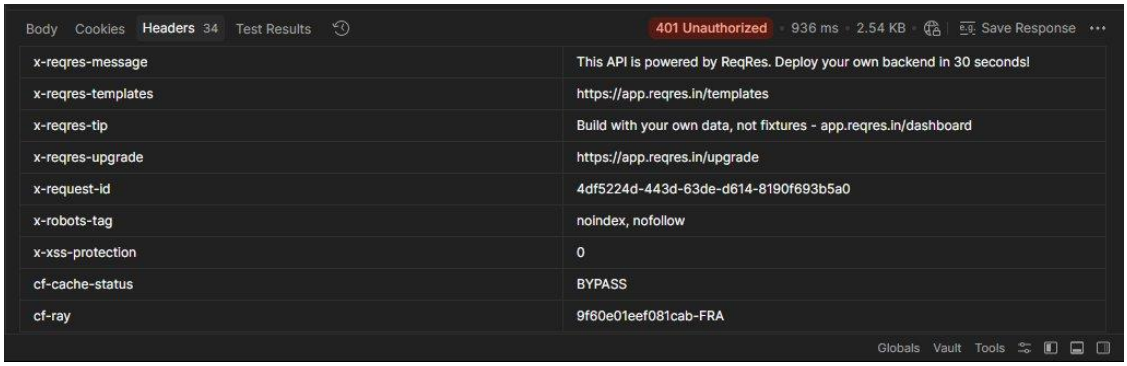

Figure 4.4 – x-reqres-message, x-reqres-templates, x-reqres-upgrade and cf-ray confirmed

4.3.2 Security Implication

While individually low-severity, header-based information disclosure provides the intelligence layer for a reconnaissance chain. Knowledge of the hosting provider, CDN, and internal URL structure allows an adversary to identify platform-specific vulnerabilities, known CVEs for the framework version, or conduct targeted phishing simulating the internal portal.

Finding 4 — Improper HTTP Method Handling on /api/login

OWASP Category	API7:2023 – Security Misconfiguration
Severity	MEDIUM
Affected Endpoint	GET /api/login
Expected Response	405 Method Not Allowed
Actual Response	**200 OK with full HTML (104.35 KB)**
CVSS v3 Score (Est.)	5.3 (Medium)

4.4.1 Observation

Sending a GET request to the /api/login endpoint — which is designed exclusively for POST operations — returns an HTTP 200 OK response with a full 104.35 KB HTML document (the Postman web interface landing page). The server does not return a 405 Method Not Allowed status, nor does it return any structured API error response.

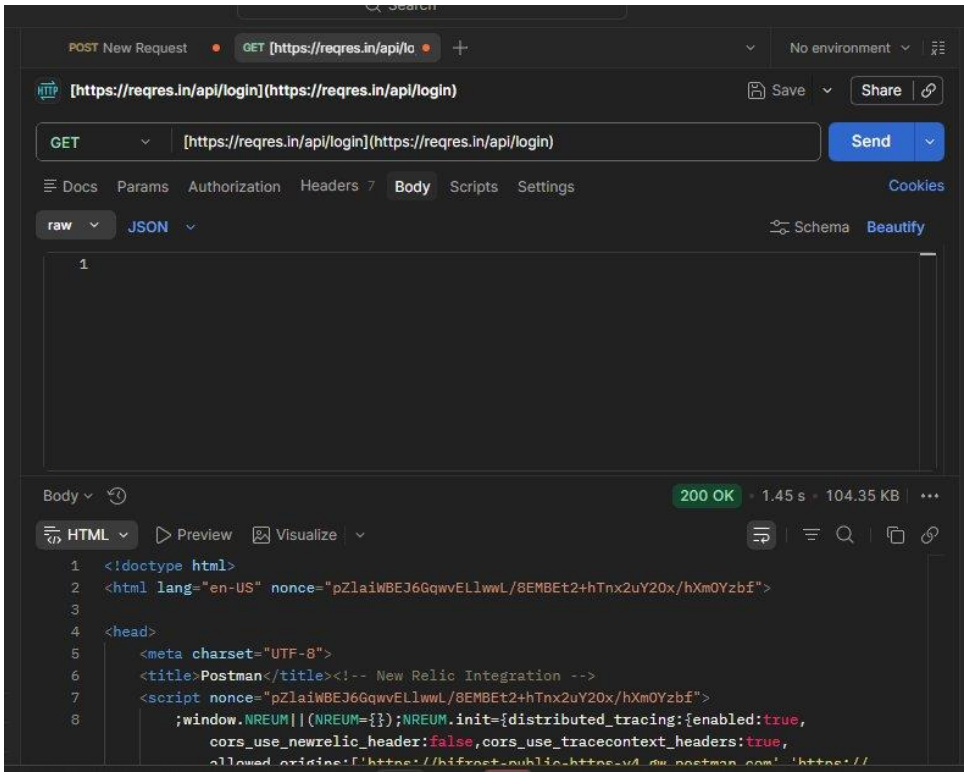

Figure 4.5 – GET /api/login returning 200 OK with HTML response body (104.35 KB)

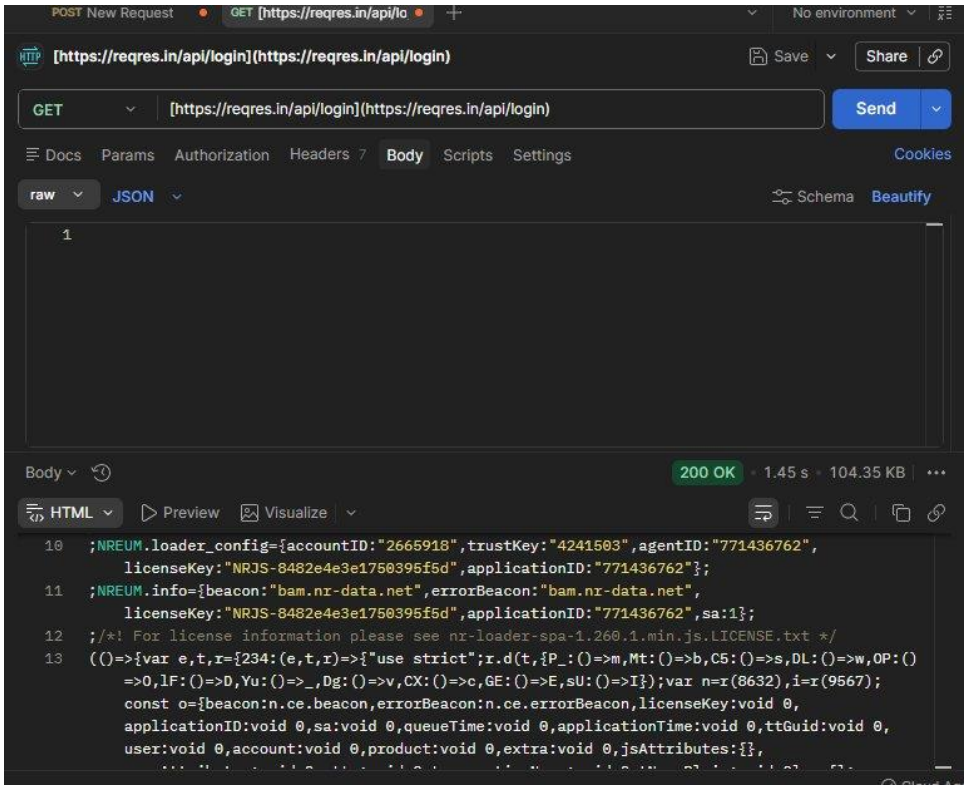

Figure 4.6 – HTML response body containing third-party JS telemetry keys (New Relic account IDs visible)

4.4.2 Security Implication

Permissive HTTP method handling creates several risks: it increases the attack surface by allowing unintended interaction patterns; the HTML response inadvertently discloses third-party telemetry configuration (New Relic account IDs, trust keys, and agent IDs visible in the returned JavaScript); and the 200 OK response may confuse API consumers and downstream monitoring tools that rely on status codes for health assessment.

Finding 5 — Credentials Transmitted Without Additional Client-Side Encryption

OWASP Category	API8:2023 – Security Misconfiguration / Improper Assets Management
Severity	LOW
Affected Endpoint	POST /api/login
Payload Format	Plaintext JSON { email, password }
Transport Encryption	TLS 1.3 (HSTS enforced)
CVSS v3 Score (Est.)	4.0 (Medium-Low)

4.5.1 Observation

Login credentials are transmitted as plaintext key-value pairs within a JSON request body. While TLS provides transport-layer encryption, there is no evidence of additional application-layer credential hardening such as password hashing, challenge-response, or token-based pre-authentication.

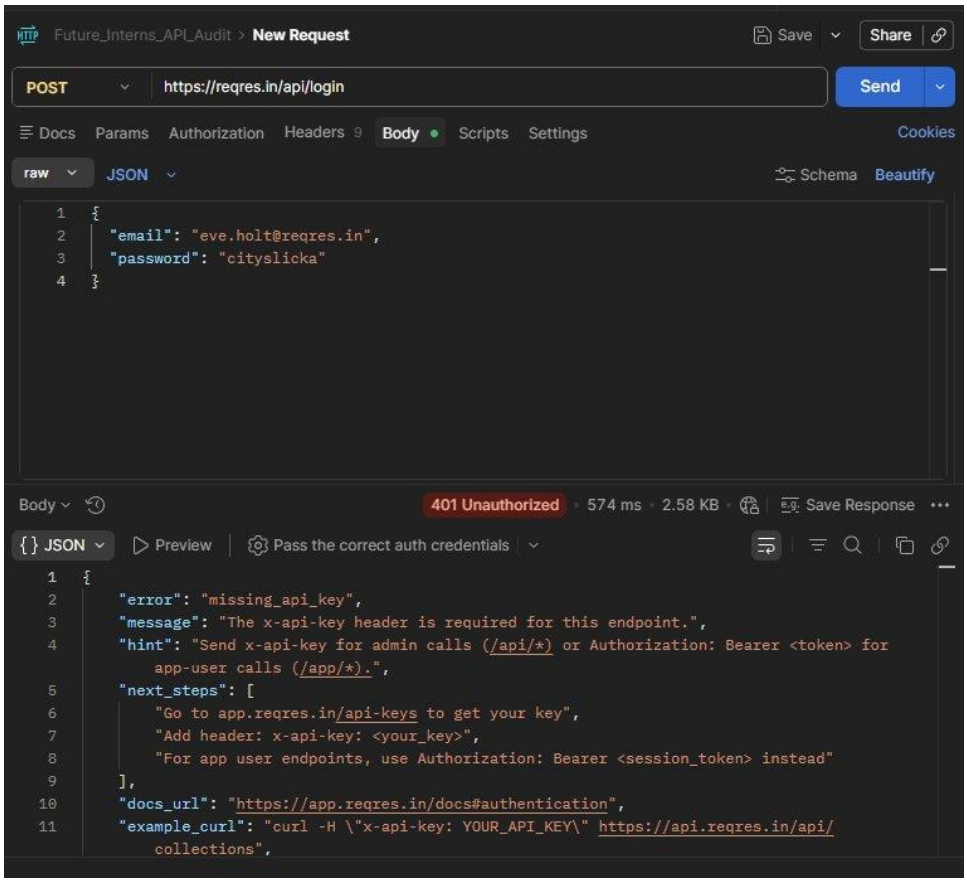

Figure 4.7 – POST /api/login with plaintext email and password in JSON body (401 returned – missing api key)

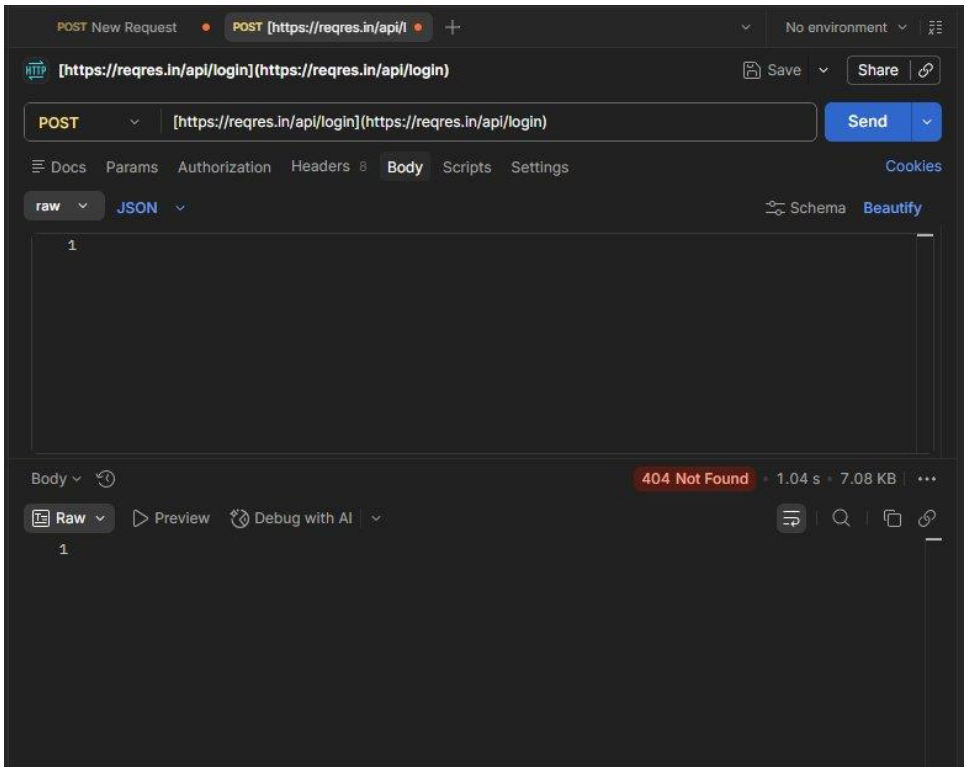

Figure 4.8 – POST /api/login body payload (404 Not Found when using malformed URL format)

4.5.2 Security Implication

Reliance solely on TLS for credential protection is acceptable under standard industry practice when TLS is correctly configured (which it is here, with HSTS enforced). However, for a production SaaS platform, plaintext credential transmission creates risk if TLS termination occurs at a proxy, if a misconfigured CDN logs request bodies, or if an insider threat can access proxy logs. Credential fields should be treated with the same sensitivity as PCI-DSS cardholder data.

5. Risk Matrix

The following risk matrix classifies each finding using the OWASP API Security Top 10 (2023 edition). Risk level is calculated by combining the likelihood of exploitation with the potential impact on a production SaaS environment.

ID	Finding	OWASP Ref	Severity	Impact	Likelihood	Risk
F-01	Broken Authentication & Verbose Errors	*API2:2023*	**HIGH**	HIGH	LOW	**HIGH**
F-02	CORS Wildcard Misconfiguration	*API7:2023*	**MEDIUM**	HIGH	MEDIUM	**MEDIUM**
F-03	Information Disclosure via Headers	*API7:2023*	**LOW**	MEDIUM	LOW	**LOW**
F-04	Improper HTTP Method Handling	*API7:2023*	**MEDIUM**	MEDIUM	MEDIUM	**MEDIUM**
F-05	Credential Plaintext Transmission	*API8:2023*	**LOW**	HIGH	LOW	**LOW**

6. Business Impact Analysis

The following analysis translates each technical finding into concrete business risk terms relevant to a SaaS organisation operating under regulatory obligations such as GDPR, SOC 2 Type II, ISO 27001, and PCI-DSS.

ID	Finding	Attack Scenario	Business Risk	Compliance Impact
F-01	**Broken Auth Disclosure**	Credential Stuffing Enablement	Verbose errors guide attackers through the exact authentication flow. In a production SaaS, this would accelerate brute-force and credential stuffing campaigns, potentially leading to account takeover at scale.	*Account takeover, customer data breach, GDPR/SOC2 violation, regulatory fines.*
F-02	**CORS Wildcard**	Cross-Site Request Forgery & Data Exfiltration	Any malicious website can initiate API calls on behalf of authenticated users. Combined with a future access-control-allow-credentials: true header (common developer error), the attacker can read authenticated API responses.	*Customer PII exfiltration, session hijacking, reputational damage, GDPR Article 32 violation.*
F-03	**Header Disclosure**	Targeted Infrastructure Attacks	Knowing the exact hosting provider (Heroku), CDN (Cloudflare), and internal URL structure allows adversaries to search for platform-specific CVEs, craft convincing phishing portals, or target the hosting provider directly.	*Pivot attacks, phishing campaigns, exploitation of known platform vulnerabilities.*
F-04	**Method Mishandling**	Information Leakage & Attack Surface Expansion	The 200 OK response to GET /api/login discloses third-party monitoring configuration (New Relic keys), and the permissive method handling may enable parameter pollution or abuse of unintended endpoint behaviours.	*Third-party service abuse, monitoring data exfiltration, API enumeration facilitation.*
F-05	**Plaintext Credentials**	Insider Threat & Proxy Log Exposure	If TLS terminates at a CDN or load balancer that logs request bodies (a common DevOps misconfiguration), credentials are exposed in plaintext logs accessible to infrastructure administrators or attackers who breach the logging system.	*Credential exposure, insider threat, compliance failure (PCI-DSS Requirement 8, NIST 800-53 IA-5).*

6.1 Aggregate Business Risk Summary

If the five findings above were to be present in a live, revenue-generating SaaS platform, the combined risk profile would represent:

- Potential for regulatory fines up to 4% of global annual turnover under GDPR Article 83(4) in the event of a data breach attributable to these weaknesses
- Loss of SOC 2 Type II certification eligibility due to failure of Security Trust Services Criteria CC6.1 (logical access controls) and CC6.6 (boundary protection)
- Reputational damage quantifiable in customer churn, increased CAC, and reduced valuation multiples
- Incident response costs estimated at $4.45M average per breach (IBM Cost of a Data Breach Report 2023) — predominantly driven by the first three findings

7. Remediation Roadmap

The following remediation roadmap is prioritised using a risk-based approach. Immediate actions address the highest-severity findings; short-term actions address systemic misconfigurations; and long-term actions establish preventive controls to eliminate classes of vulnerability from the development lifecycle.

7.1 Immediate Actions (0–7 Days) — HIGH Severity

F-01 Fix: Sanitise Error Responses

- Replace verbose authentication error bodies with a single generic message: { "error": "Unauthorised" }
- Remove all hint, next_steps, docs_url, example_curl, and _meta fields from error responses
- Implement a global error handler/middleware that strips internal details before serialisation
- Ensure all 4xx/5xx responses return identical structures regardless of root cause
- Test with OWASP ZAP in passive scan mode to verify no sensitive data leaks in error responses

Before (Vulnerable)	After (Fixed)
`{ "error": "missing_api_key", "hint": "...", "docs_url": "..." }`	`{ "error": "Unauthorised", "status": 401 }`

7.2 Short-Term Actions (7–30 Days) — MEDIUM Severity

F-02 Fix: Restrict CORS to Approved Origins

- Replace Access-Control-Allow-Origin: * with an explicit allowlist of trusted origins
- Use environment-specific values: production domain(s) only in production, localhost only in development
- Never combine Access-Control-Allow-Credentials: true with a wildcard origin (browsers block this but it is frequently misconfigured in proxies)
- Example Express.js implementation:

```
const corsOptions = {
  origin: ['https://your-saas-app.com', 'https://admin.your-saas-app.com'],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
};
app.use(cors(corsOptions));
```

F-04 Fix: Enforce HTTP Method Restrictions

- Define and enforce an allowed-methods list at the API gateway or router level for every endpoint
- Return 405 Method Not Allowed with an Allow response header listing permitted methods
- Ensure the router does not fall through to a catch-all that serves HTML content
- Example implementation: app.get('/api/login', (req, res) => res.status(405).set('Allow', 'POST').json({ error: 'Method not allowed' }))

7.3 Long-Term Actions (30–90 Days) — Systemic Controls

F-03 Fix: Strip Sensitive Response Headers

- Remove or sanitise X-Powered-By at the web server/proxy level (nginx: more_clear_headers 'X-Powered-By')
- Remove all x-reqres-* custom informational headers — these serve no security purpose and should be internal-only
- Ensure server header returns a generic value (e.g., server: web) not the specific CDN/PaaS name
- Implement a security-headers middleware that explicitly sets the secure headers policy on every response

F-05 Fix: Harden Credential Handling

- Ensure logging pipelines (CDN, proxy, APM) are configured to never log request bodies containing password fields
- Implement SRP (Secure Remote Password) or OPAQUE protocol for zero-knowledge password authentication on sensitive endpoints
- Consider moving to OAuth 2.0 with PKCE for all client authentication — eliminates password transmission entirely
- Audit all third-party integrations (CDN, monitoring) for request body logging configurations

Global Header Security Policy — Recommended Secure Header Set

Header	Recommended Value
Strict-Transport-Security	*max-age=63072000; includeSubDomains; preload*
Content-Security-Policy	*default-src 'self'; script-src 'self'; object-src 'none'*
X-Frame-Options	*DENY*
X-Content-Type-Options	*nosniff*
Referrer-Policy	*no-referrer*
Permissions-Policy	*camera=(), microphone=(), geolocation=()*
X-XSS-Protection	*1; mode=block (or omit in favour of CSP)*
Access-Control-Allow-Origin	*https://your-domain.com (explicit, never *)*

8. Conclusion

This read-only security audit of the ReqRes public API platform surfaced five actionable findings across the OWASP API Security Top 10 framework. The most critical finding — verbose authentication error disclosure — directly accelerates the reconnaissance phase of an attack and should be addressed within the first week of remediation. The CORS wildcard and improper method handling findings, while medium severity individually, create compounding risk when combined with the information disclosure finding.

It is important to note that ReqRes is a public testing platform, and some of these design choices may be intentional for educational purposes. However, the same patterns in a production SaaS environment would represent material security risk to the organisation, its customers, and its compliance posture.

The remediation roadmap provided in Section 7 is structured to deliver immediate risk reduction within the first week, systematic misconfiguration remediation within 30 days, and long-term defence-in-depth controls within 90 days. Following these recommendations will significantly raise the cost and complexity for any adversary attempting to compromise the API layer.

8.1 Auditor Attestation

Prepared By	Prasad Sarkate
Programme	Future Interns – Cyber Security Internship
Task	Task 3 – API Security Audit
Date	26 April 2026
Status	FINAL
Signature	_____